A Domain-Specific Language and Web Interface for Geometric Simplicial Complexes in PolyLogicA

Simone Riccio

July 14, 2026

Abstract

This report presents the design and implementation of a novel Domain-Specific Language (DSL) and web interface tailored for the construction and manipulation of geometric simplicial complexes. Developed to interface directly with the PolyLogicA branch of the VoxLogicA project, this tool allows users to define topological spaces in Euclidean coordinates, apply geometric transformations, and evaluate spatial logic queries. We detail the language grammar, the abstract syntax tree evaluation, and the topological serialization process required to generate the Hasse diagram (poset) representations necessary for spatial model checking.

1 Introduction

2 Theoretical Background

2.1 Geometry and Simplicial Complexes

Definition 1 (Simplex)**.** *Given a set of $k+1$ affinely independent points in $\mathbb{R}$ denoted as $\{v_0, v_1, \ldots, v_k\}$, the k-simplex σ spanned by these points is defined as the convex hull:*

$$\sigma = [v_0, v_1, \ldots, v_k] = \left\{ \sum_{i=0}^{k} \lambda_i v_i \mid \lambda_i \geq 0, \sum_{i=0}^{k} \lambda_i = 1 \right\}.$$

Definition 2 (Simplicial Complex)**.** *A simplicial complex K is a finite collection of simplices such that:*

1. *Every face of a simplex in K is also in K.*
2. *The intersection of any two simplices in K is either empty or a face of both simplices.*

[Image Placeholder]: Immagini di simplessi e complessi simpliciali di varie dimensioni

Remark 1. *Simplicial complexes provide a combinatorial representation of topological spaces, allowing for the application of algebraic topology techniques and spatial logic reasoning.*

Definition 3 (Abstract Simplicial Complex)**.** *An abstract simplicial complex is a set K of finite subsets of a vertex set V such that if $\sigma \in K$ and $\tau \subseteq \sigma$, then $\tau \in K$. The elements of K are called simplices, and the elements of V are called vertices.*

Definition 4 (Dimension)**.** *The dimension of a simplex $\sigma = [v_0, v_1, \ldots, v_k]$ is defined as k. The dimension of a simplicial complex K is the maximum dimension of its simplices:*

$$\dim(K) = \max\{\dim(\sigma) \mid \sigma \in K\}.$$

Definition 5 (Downward Closure)**.** *The downward closure of a simplex σ is the set of all its faces:*

$$C(\sigma) = \{\tau \subseteq \sigma \mid \tau \text{ is a face of } \sigma\}.$$

2.2 Posets and Hasse Diagrams

Definition 6 (Partially Ordered Set (Poset))**.** *A partially ordered set (poset) is a set P equipped with a binary relation $\leq$ that is reflexive, antisymmetric, and transitive.*

Definition 7 (Hasse Diagram)**.** *A Hasse diagram is a graphical representation of a finite poset, where each element is represented as a vertex, and edges are drawn to indicate the covering relations. An edge from vertex a to vertex b is drawn if $a < b$ and there is no c such that $a < c < b$. The diagram is typically drawn so that if $a < b$, then b appears higher than a in the diagram.*

Remark 2. *The set of simplices in a simplicial complex, ordered by inclusion, forms a poset. The Hasse diagram of this poset visually represents the inclusion relationships among the simplices, which is crucial for spatial model checking in PolyLogicA.*

[Image Placeholder]: Immagini di un complesso simpliciale a sinistra e il suo diagramma di Hasse a destra

Remark 3. *The transition from geometric simplicial complexes to partially ordered sets (posets) is the core computational step for interfacing with spatial model checkers like PolyLogicA. While a simplex is defined by its coordinates in Euclidean space, its logical existence in the model checker is defined by its position in a hierarchy of inclusion.*

2.3 Spatial Logic

Broadly defined, a spatial logic is any formal language interpreted over a class of structures featuring goemetrical entities and relations. [1] provides a comprehensive overview of the various spatial logics, their semantics, and applications.

The origin of this field lies in the topological approach to modal logic initiated by Alfred Tarski, who recognized that modal operators could represent topological notions such as interior and closure.

Modern developments in spatial logic extend these ideas to discrete spatial structures (like graphs or images) using a generalized topological space called a **closure space** [2]

Definition 8 (Closure Space)**.** *A closure space is a pair $(X, \mathcal{C})$ where X is a set and $\mathcal{C} : \mathcal{P}(X) \to \mathcal{P}(X)$ is a closure operator satisfying the following properties for all $A, B \subseteq X$:*

1. $\mathcal{C}(\emptyset) = \emptyset$
2. $A \subseteq \mathcal{C}(A)$ *(Extensivity)*
3. *If $A, B \subseteq X$ and $\mathcal{C}(A \cup B) = \mathcal{C}(A) \cup \mathcal{C}(B)$*

Remark 4. *In any topological space X we can define $\mathcal{C}(A)$ as the topological closure of A, which is the smallest closed set containing A. This closure operator satisfies the properties of a closure operator, making $(X, \mathcal{C})$ a closure space.*

The closure operator in this case has one additional property: it is idempotent, meaning that $\mathcal{C}(\mathcal{C}(A)) = \mathcal{C}(A)$ for all $A \subseteq X$. This property is not required for a closure space, but it is a defining characteristic of topological spaces.

Example 1 (A closure space which is not a topological space)**.** *Consider for a oriented graph $G = (V, E)$ the closure operator $\mathcal{C}$ defined as follows: for any $A \subseteq V$, $\mathcal{C}(A)$ is the set of all vertices in V that can be reached from any vertex in A by a directed path in G. This closure operator satisfies the properties of a closure operator, making $(V, \mathcal{C})$ a closure space.*

However, this closure operator is not necessarily idempotent. For example, if A consists of a single vertex with outgoing edges to other vertices, then $\mathcal{C}(A)$ will include those reachable vertices. Applying $\mathcal{C}$ again may yield additional vertices that are reachable from the newly included

vertices, thus $\mathcal{C}(\mathcal{C}(A))$ may be strictly larger than $\mathcal{C}(A)$. Therefore, $(V, \mathcal{C})$ is a closure space but not a topological space.

If the graph is undirected, then the closure operator becomes idempotent, and $(V, \mathcal{C})$ is a topological space.

Ciancia et al. define a spatial logic for closure spaces for reasoning about spatial properties of closure spaces.

Definition 9 (Spatial Logic for Closure Spaces)**.** *[2] In a closure space $(X, \mathcal{C})$, the syntax of the spatial logic is defined as follows:*

$$\phi ::= p \mid \neg\phi \mid \phi_1 \wedge \phi_2 \mid \mathcal{N}\phi \mid \phi_1 \mathcal{U} \phi_2$$

where p is a propositional variable, $\neg$ is negation, $\wedge$ is conjunction, $\mathcal{N}$ is the "next" operator, and $\mathcal{U}$ is the "until" operator. The semantics of the logic is defined in terms of the closure operator $\mathcal{C}$ and the satisfaction relation $\models$ as follows:

- $(X, \mathcal{C}), x \models p$ *if* $x \in V(p)$*, where $V(p)$ is the set of points in X where p is true.*
- $(X, \mathcal{C}), x \models \neg\phi$ *if* $(X, \mathcal{C}), x \not\models \phi$*.*
- $(X, \mathcal{C}), x \models \phi_1 \wedge \phi_2$ *if* $(X, \mathcal{C}), x \models \phi_1$ *and* $(X, \mathcal{C}), x \models \phi_2$*.*
- $(X, \mathcal{C}), x \models \mathcal{N}\phi$ *if there exists* $y \in \mathcal{C}(\{x\})$ *such that* $(X, \mathcal{C}), y \models \phi$*.*
- $(X, \mathcal{C}), x \models \phi_1 \mathcal{U} \phi_2$ *if there exists a finite sequence of points* $x_0, x_1, \ldots, x_n \in X$ *such that* $x_0 = x$*,* $(X, \mathcal{C}), x_n \models \phi_2$*, and for all* $0 \leq i < n$*,* $(X, \mathcal{C}), x_i \models \phi_1$*.*

We will not go into the details of the semantics of the logic, but we will focus on how to represent geometric simplicial complexes in a way that allows us to reason about their spatial properties using this logic.

3 Geometric DSL

3.1 Structure

The DSL is designed to be a simple and expressive language for defining geometric simplicial complexes. The DSL's code is structured into several components, each responsible for a specific aspect of the language's functionality. The main components are:

- **Parser**: The parser is responsible for reading the DSL code and converting it into an abstract syntax tree (AST).
- **Core**: The core component is responsible for evaluating the AST and constructing the corresponding geometric simplicial complex.
- **Exporter**: The exporter component is responsible for serializing the geometric simplicial complex into a format that can be used by the web interface and by PolyLogicA.

They are implemented in the following files:

Listing 1: Project directory tree

```
project-root/
|-- report/
|   `-- main.tex
|-- src/
|   |-- parser.py
```

```
|   |-- core.py
|   `-- exporter.py
|-- web/
|   |-- index.html
|   |-- render.js
    `-- visualization.js
`-- README.md
```

3.2 Syntax

The syntax of the DSL is deisegned for defining geometric simplicial complexes in a concise manner. The language allows users to specify vertices, simplices, some operations on them.

Listing 2: Grammar of the Geometric DSL

```
    program: statement*

    ?statement: point_decl
              | complex_decl
              | assign
              | render_stmt
              | func_decl
              | return_stmt

    point_decl: "point" IDENT "=" tuple_coord
    complex_decl: "complex" IDENT "=" expr
    render_stmt: "render" IDENT
    func_decl: "function" IDENT "(" param_list ")" "{{" statement* "}}"
    return_stmt: "return" expr

    ?expr: op_call
        | func_call
        | IDENT
        | vertices_list
        | tuple_coord
        | NUMBER
        | "(" expr ")"

    op_call: OP "(" arg_list? ")"
    func_call: expr "(" call_args ")"

    tuple_coord: "(" expr "," expr ")"
    vertices_list: "[" id_list "]"
    id_list: IDENT ("," IDENT)*
    arg_list: expr ("," expr)*
    param_list: (IDENT ("," IDENT)*)?
    call_args: (expr ("," expr)*)?

    OP.2: /{op_regex}/
    IDENT: /[A-Za-z_][A-Za-z0-9_]*/

    NUMBER: /-?[0-9]+(\.[0-9]+)?/

    COMMENT: "//" /[^\n]/* "\n"
    %ignore COMMENT
    WS: /[ \t\f\r\n]+/
    %ignore WS
```

To parse this grammar, we use the Lark parser library in Python in the file `dsl_parser.py`. The parser generates an abstract syntax tree (AST) that represents the structure of the input program. The AST is then traversed to evaluate the expressions and construct the corresponding geometric simplicial complex.

- **Point Declarations** (`PointDecl`): Instantiates a coordinate pair (x, y) in $\mathbb{R}^2$.
- **Complex Declarations** (`ComplexDecl`): Instantiates a simplicial complex from a list of vertices or through operations.
- **Render Statements** (`RenderStmt`): Registers a target complex to be exported and visualized by the web interface.
- **Function Definitions** (`FunctionDecl`): Standard functional structures that yield runtime closures to enable procedural abstraction.

3.3 Semantics: Evaluation and Denotational Values

The evaluation of the program is defined over well-defined expressible and denotationalvalues:

Listing 3: Types for Evaluation

```
type EVal = Geometric | Num | Closure | None
type DVal = EVal | Operator | Closure

type Geometric = Point | Complex
type Num = int | float
```

The variable evluation enviroment is a mapping from identifiers to denotational values:

$$\rho : \text{IDENT} \to \text{DVal}$$

It is implemented as a Dictionary in Python, where the keys are the identifiers and the values are the corresponding denotational values.

Listing 4: Environment

```
type Environment = Dict[str, DVal]
```

The evaluation of the program is defined as a function that takes an AST and returns a environment.

3.4 Semantics: Operations

The language supports a set of operations that can be applied to geometric simplicial complexes. These operations are defined as functions that take complexes and numbers as input and return a new complex as output. The operations include:

Listing 5: Geometric Operations

```
def union(complex1: Complex, complex2: Complex) -> Complex:

def intersection(complex1: Complex, complex2: Complex) -> Complex:

def difference(complex1: Complex, complex2: Complex) -> Complex:

def translate(complex: Complex, vector: Tuple[float, float]) ->
    Complex:

def scale(complex: Complex, factor: float) -> Complex:
```

The semantics of these operations are defined in terms of the underlying geometric representation, therefore we will not go into the details making the assumption that the reader is familiar with basic geometric operations.

There are also some other operations which maybe have a more complex semantics, such as

Listing 6: Topological Operations

```
    def boundary(complex: Complex) -> Complex:
        """Returns the boundary of the complex"""

    def star(complex: Complex, vertex: Point) -> Complex:
        """Returns the star of a vertex in the complex"""
```

- **Boundary (∂):** For a simplicial complex K, the boundary operator computes its topological boundary. Geometrically, if K is a d-dimensional simplicial complex, its boundary consists of those $(d-1)$-simplices that are faces of exactly one d-simplex, along with their downward closure:

  $$\partial(K) = \{\sigma \in K \mid \dim(\sigma) < \dim(K) \text{ and } \sigma \text{ is a face of an odd number of maximal simplices}\}.$$

  In our constructive implementation, it extracts this sub-complex, preserving the combinatorial and structural requirements of a simplicial complex.

- **Star (St):** Given a simplicial complex K and a target vertex (or sub-complex) v, the star consists of all simplices in K that contain v:

  $$\mathrm{St}(v) = \{\sigma \in K \mid v \subseteq \sigma\}.$$

  Since the open star is not necessarily closed under taking subset faces, our DSL implementation returns the **closed star** (the closure of the star), which automatically includes all subset faces, guaranteeing that the result forms a valid, self-contained sub-complex.

These have been implemented because one of the future goals of this project would be to reason about algebraic topological concepts such as holes.

3.5 Semantics: Render Operation

Another operation is the `render` operation, which is used to mark a complex for export and visualization. The semantics of this operation is to add the complex to a list of render targets, which will be serialized and sent to the web interface for visualization.

4 Implementation Details

The execution pipeline of the DSL consists of three main phases: parsing the source text into an Abstract Syntax Tree (AST), dynamically evaluating the AST within a scoped environment, and serializing the resulting geometric structures into a partially ordered set (poset) for PolyLogicA.

4.1 Core Data Structures

To guarantee mathematical correctness, especially regarding the properties of simplicial complexes (where order of vertices does not matter and duplicates are not allowed), we leverage Python's native data structures.

In `core.py`, a `Point` is defined as a dataclass, ensuring it is hashable. A `Complex` is represented as a Python `Set` of `FrozenSet`s of Points:

Listing 7: Core Geometric Data Structures

```
@dataclass(frozen=True)
class Point:
    x: float
    y: float

@dataclass
class Complex:
    simplices: Set[FrozenSet[Point]]
```

This specific formulation guarantees that operations like union and intersection can directly leverage highly optimized set-theoretic operations at the Python interpreter level.

4.2 AST Evaluation and Control Flow

The evaluator traverses the AST nodes defined in `parser.py` (e.g., `PointDecl`, `ComplexDecl`, `FuncCall`) and recursively resolves them against the active `Environment`.

A notable implementation detail is the handling of function calls and lexical closures. When a `FunctionDecl` is evaluated, it is packed into a `Closure` object that captures the current environment. To manage control flow—specifically early exits via `return` statements inside function bodies—we utilize a custom exception mechanism:

Listing 8: Exception-based Control Flow for Returns

```
class ReturnException(Exception):
    def __init__(self, value: EVal):
        self.value = value
```

When the evaluator encounters a `ReturnStmt`, it raises a `ReturnException` containing the evaluated expression. The function application wrapper catches this exception and extracts the value, cleanly simulating call-stack unwinding.

4.3 Topological Serialization and Poset Generation

The transition from continuous geometric representations to discrete logical structures occurs in the `export.py` module. PolyLogicA requires topological spaces to be represented as Hasse diagrams (posets) of closure spaces.

To achieve this, we used a `_build_downward_closure` algorithm. For every complex marked by a `render_targets` directive, the algorithm extracts all maximal simplices and systematically generates all possible sub-simplices (faces, edges, vertices) using Python's `itertools.combinations`.

Listing 9: Downward Closure Generation

```
def _build_downward_closure(complexes: ComplexEnv) -> tuple[list[
    frozenset], dict[frozenset, str], dict[frozenset, set[str]]]:
    """Generates all sub-simplices, assigning unique string IDs and
       tracking their source variable atoms."""

    for name, complex_obj in complexes.items():
        for max_simplex in complex_obj.simplices:
            pts = list(max_simplex)
            # Generate all dimensional subsets (vertices, edges, faces)
            for r in range(1, len(pts) + 1):
                for sub_combo in itertools.combinations(pts, r):
                    simplex_atoms[frozenset(sub_combo)].add(name)

    sorted_simplices = sorted(simplex_atoms.keys(), key=len)
```

```
    simplex_to_id = {simp: str(idx) for idx, simp in enumerate(
        sorted_simplices)}
    return sorted_simplices, simplex_to_id, simplex_atoms
```

4.4 Integration with PolyLogicA

The serialized poset is then formatted into a JSON structure compatible with PolyLogicA's input requirements. Each simplex is assigned a unique identifier, and the covering relations are encoded as adjacency lists. This allows the spatial model checker to interpret the geometric structures and evaluate spatial logic queries effectively.

The function `export_to_polylogica` in `export.py` handles this final step, ensuring that the output is both syntactically and semantically valid for PolyLogicA's processing pipeline.

5 Examples and Model Checking

Prova

References

[1] Marco Aiello, Ian Pratt-Hartmann, and Johan van Benthem, *Handbook of Spatial Logic*, Springer, 2007.

[2] Vincenzo Ciancia et al., *VoxLogicA: A Spatial Model Checker*.